Product Overview

Обновлено 29 мая 2026, 00:41 Андрей Коновалов

Purpose

Mini Tournament Hub Backend serves short live tournaments where participants answer random round prompts with text, then sequentially vote on answers from other participants.

The MVP is designed for one tournament mode:

- multi-round;
- text prompt;
- text submission;
- no elimination;
- sequential voting;
- cumulative leaderboard.

Roles

Authenticated user

Can:

- register, log in, refresh a JWT pair, and log out;
- create a tournament if they do not own another unfinished tournament;
- view the list of draft tournaments;
- view draft tournament details;
- join an available tournament;
- get the full tournament snapshot only if they are a participant;
- connect to a Socket.IO room only if they are a participant.

Tournament participant

Can:

- submit or update one text submission in the current round;
- vote for the currently revealed submission from another participant;
- leave the tournament only while it is in `DRAFT` status and only if they are not the owner;
- participate in only one `ACTIVE` tournament at a time.

Tournament owner

Can:

- create a public or private tournament;
- start a draft tournament when there are at least 4 participants;
- cancel a draft tournament;
- get an `inviteToken` for a private tournament.

Cannot:

- leave their own tournament through the leave endpoint;
- create a second `DRAFT` or `ACTIVE` tournament.

MVP Scope

Included:

- JWT auth: registration, login, refresh, logout;
- public/private tournaments;
- invite token for private tournaments;
- draft tournament discovery;
- owner auto-participation;
- draft join/leave/cancel;
- tournament start;
- persisted bilingual round prompts;
- text submissions;
- sequential voting;
- timeout `DISLIKE` votes;

- cumulative leaderboard;
- automatic next round creation;
- automatic tournament completion;
- Socket.IO live events;
- Redis-backed presence;
- REST recovery endpoints.

Not included:

- spectator mode;
- chat, comments, reactions;
- admin/moderator roles;
- anti-fraud;
- notifications;
- file/image/meme submissions;
- editable tournament parameters after creation;
- event replay/history;
- horizontal Socket.IO scaling;
- advanced analytics.

Main User Flow

```
Register/Login
  -> Create or join draft tournament
  -> Socket.IO tournament:join
  -> Owner starts tournament
  -> Round SUBMISSION
  -> Participants submit text answers
  -> Round VOTING
  -> Server reveals one submission
  -> Participants vote LIKE/DISLIKE
  -> Server finalizes current submission
```

```
   -> Next submission or round completed
   -> Next round or tournament completed
```

Frontend Recovery Contract

After reconnecting, the client must:

1. reconnect to Socket.IO with an access token;
2. emit `tournament:join`;
3. call `GET /api/v1/tournaments/:id/full`;
4. build the UI from the REST snapshot as the authoritative state.

The backend does not replay missed realtime events.